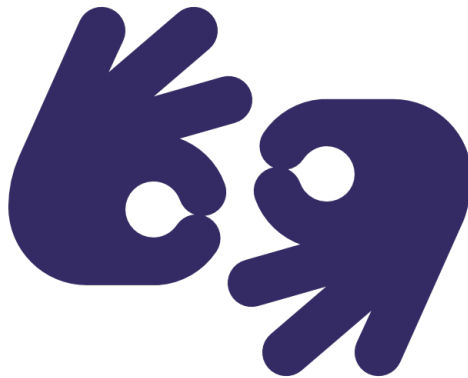


SignVR

Traduzione in tempo reale
da Inglese ad American Sign Language
in Realtà Mista

Elena Giunta,
Davide Leotta,
Damiano Rinaldi

Febbraio 2026



Indice

1	Introduzione	4
2	Disabilità Uditiva e American Sign Language	4
2.1	L’Impatto della disabilità uditiva sulla popolazione globale	4
2.2	Struttura e funzionamento dell’ <i>American Sign Language</i>	5
3	Tecnologie e Datasets	5
3.1	Meta Quest 3	5
3.2	Unity	6
3.3	Meta XR Interaction SDK	6
3.4	Meta Building Blocks	7
3.5	Meta Voice SDK	7
3.6	WIT.ai	8
3.7	LLaMA	9
3.8	Piattaforma di Inferenza: Groq	9
3.9	Mixamo	10
3.10	Affinity	10
3.11	Blender	11
3.12	<i>Animation Datasets</i>	11
3.12.1	Provenienza delle animazioni	11
3.12.2	Normalizzazione e rielaborazione	11
3.12.3	Limitazioni tecniche nell’implementazione delle animazioni . .	12
3.12.4	Implementazione limitata del vocabolario <i>ASL</i>	12
4	Il software	12
4.1	Introduzione	12
4.2	L’interfaccia	14
4.2.1	<i>Speech-To-Text</i> e <i>Gloss Notation</i>	14
4.2.2	Interprete Virtuale	14
4.2.3	Impostazioni e personalizzazione dell’esperienza utente	15
4.2.4	Sezioni <i>Tutorial</i> e <i>Credits</i>	16
4.2.5	Implementazione dell’interfaccia utente	17
4.3	Implementazione del pannello delle impostazioni e salvataggio delle <i>PlayerPrefs</i>	18
4.3.1	Gestione dell’interfaccia, <i>SceneManager</i> e chiusura dell’appli- cazione	23
4.4	Gestures	25

4.4.1	Riconoscimento delle mani	25
4.4.2	Implementazione delle gestures	25
4.5	Implementazione al supporto per controller Meta Quest	26
4.6	Implementazione dello <i>Speech-To-Text</i>	26
4.6.1	Periodo di ascolto e soglia di silenzio	27
4.7	Implementazione della trascrizione in <i>Gloss</i>	27
4.7.1	Ricezione del testo e <i>VoiceDictationManager</i>	28
4.7.2	Definizione del <i>Prompt</i> per la trascrizione in <i>Gloss</i>	29
4.7.3	Gestione dell' <i>LLM</i> attraverso <i>LlmTranslationHandler</i>	32
4.8	Animazioni e interpreti virtuali	33
4.8.1	Standardizzazione e interpolazione delle animazioni	33
4.8.2	<i>AnimationHandler</i> come coordinatore delle animazioni	33
4.9	<i>Lazy-Follow</i> e movimento dell'interfaccia	36
4.9.1	Attributi dello <i>script</i> e definizione del comportamento	37
4.9.2	Avvio dello <i>script</i> e <i>ForceUpdatePosition()</i>	37
4.9.3	Funzione di <i>LateUpdate()</i> , controllo della posizione e movimento dell'interfaccia	38
4.9.4	Gestione della rotazione tramite <i>HandleRotation()</i>	39
5	Note conclusive	40

1 Introduzione

SignVR è un *software* di Realtà Mista per la **traduzione in tempo reale** dalla lingua Inglese alla Lingua dei Segni Americana. Il progetto si pone sia come strumento comunicativo per persone affette da disabilità uditiva, fornendo un **interprete personale** all'utente, che come strumento didattico di primo approccio alla lingua dei segni e notazione *Gloss*. Il sistema, sviluppato in Unity per Meta Quest 3, si divide in tre componenti fondamentali: un modulo di *Speech-To-Text*, con conseguente trascrizione in *Gloss*, un assistente virtuale che interpreta il parlato in *American Sign Language* ed un pannello di impostazioni che permette di personalizzare l'esperienza utente.



Figura 1: *Combination Mark* di **SignVR**, il simbolo affiancato al testo rappresenta il termine "Interprete" in *American Sign Language*.

2 Disabilità Uditiva e American Sign Language

2.1 L'Impatto della disabilità uditiva sulla popolazione globale

La disabilità uditiva è una limitazione che impatta quotidianamente la vita di **430 milioni di persone**; il dato è destinato a salire: si stima che il 10% della popolazione globale, entro il 2050, avrà una forma invalidante di perdita d'udito.[21]¹ La perdita d'udito è la forma più diffusa di disabilità sensoriale[22][16], la popolazione coinvolta ha difficoltà maggiori in ambito comunicativo, è più predisposta all'isolamento e alla stigmatizzazione sociale, può avere maggiori difficoltà in contesti lavorativi e vede esiti generalmente peggiori nei temi di salute fisica e mentale.[15][21]

¹Una perdita d'udito superiore ai 35dB nell'orecchio minormente compromesso è considerata invalidante[21].

2.2 Struttura e funzionamento dell'*American Sign Language*

L'*American Sign Language* è il linguaggio visuale di riferimento per la popolazione sorda degli Stati Uniti d'America² e del Canada anglofono[5][13]. Il linguaggio si basa sui seguenti elementi:

- *Handshape*: forma assunta dalla mano (dita tese, mano chiusa a pugno ecc.);
- *Orientation*: direzione su cui punta il palmo della mano (parallelamente al corpo dell'interprete, verso il corpo dell'interprete ecc.);
- *Location*: da dove viene eseguito il movimento rispetto il corpo dell'interprete (fronte, petto ecc.);
- *Movement*: se il movimento viene ripetuto, ha un andamento circolare o viene eseguito una sola volta;
- *Non-Manual Expressions*.³: mimica facciale, movimenti di sopracciglia e bocca, rotazione del capo

3 Tecnologie e Datasets

Per lo sviluppo di **SignVR** sono state utilizzate diverse tecnologie e incrociate le animazioni di due *datasets*, segue un elenco di ciò che ha reso possibile lo sviluppo del software.

3.1 Meta Quest 3

Terza generazione della linea *Meta Quest*⁴, è un dispositivo VR *all-in-one* lanciato sul mercato ad ottobre 2023[4] Meta è l'attuale leader di mercato nel settore dei *VR Headsets* con un *Market Share* del 74.6%[7] e garantisce la *Forward-Compatibility* per le successive generazioni di dispositivi *Horizon OS*,[9] rendendola una importante piattaforma di sviluppo.

²Il 3% della popolazione Americana conosce l'*ASL*, l'83% della quale è popolazione udente.

³le *NME* vengono utilizzate principalmente per punteggiatura, intonazione e grammatica.

⁴originariamente chiamato *Oculus Quest*.



Figura 2: Immagine promozionale per il *Meta Quest 3*.

3.2 Unity

È una piattaforma di sviluppo *Cross-Platform* impiegata nello sviluppo di giochi digitali, simulatori ed esperienze interattive.[19] In ambito XR, **Unity** è il *Game Engine* prevalente: è stato utilizzato per sviluppare oltre il 70% dei giochi più venduti nel *Quest Store*[20]⁵ ed è generalmente considerata una piattaforma accessibile, flessibile e ben ottimizzata rispetto l'uso delle risorse e requisiti *hardware*[2].

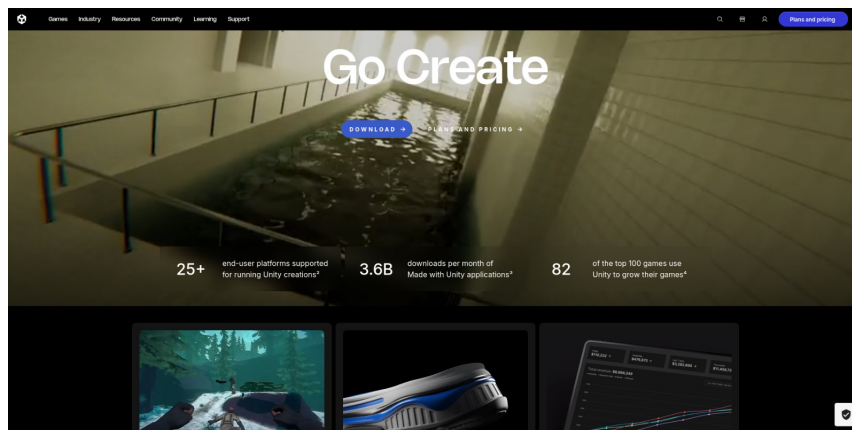


Figura 3: *Homepage* del sito *Unity.com*.

3.3 Meta XR Interaction SDK

È uno *Standard Development Kit* fornito da *Meta* per aggiungere interazioni con *controller*, mani e *body pose*[12]. Il suo impiego in permette di testare ed implementare rapidamente le *features* interattive dell'applicazione.

⁵ *Marketplace* di riferimento per applicazioni native **Horizon OS**.

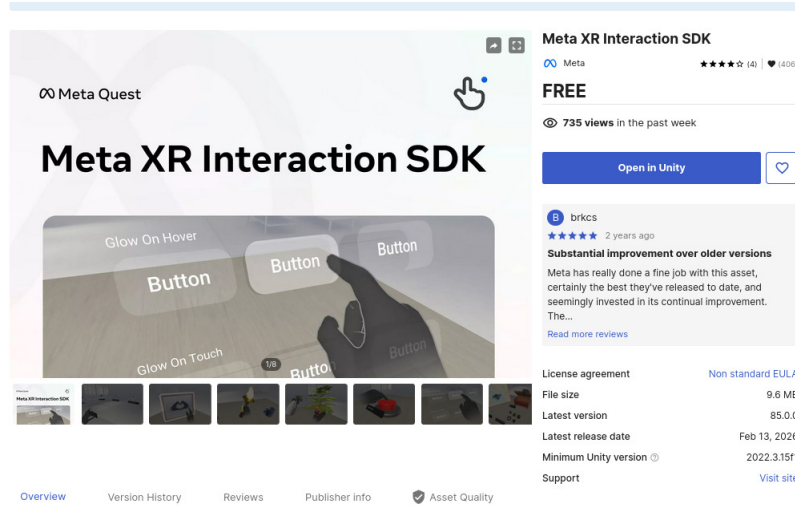


Figura 4: *Meta XR Interaction SDK* nello *Unity Asset Store*.

3.4 Meta Building Blocks

È un'estensione di **Unity** fornita da Meta Platforms, Inc. per velocizzare lo sviluppo e facilitare l'utilizzo di funzionalità dei dispositivi *Meta Quest*[10]. L'utilizzo dei **building blocks**, definiti da Meta come «unità atomiche delle funzionalità *Meta Quest*» ha favorito lo sviluppo di **SignVR** semplificando e velocizzando l'implementazione di nuove *features*.

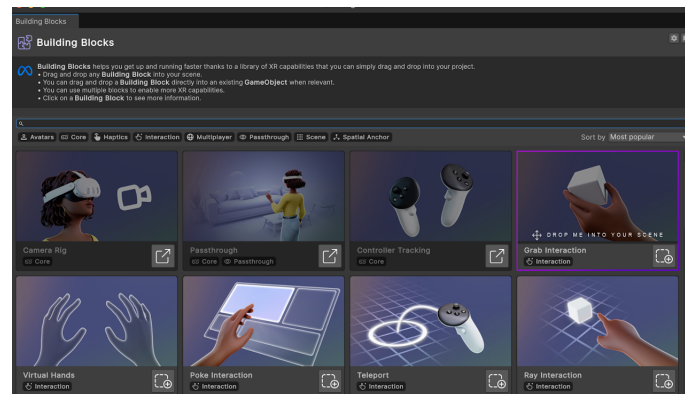


Figura 5: Interfaccia **Building Blocks** in **Unity**.

3.5 Meta Voice SDK

È uno *Standard Development Kit* sviluppato da Meta Platforms Inc. per l'implementazione dei comandi vocali[11]. Viene utilizzato in **SignVR** per permettere lo *Speech-To-Text*, grazie alla sua integrazione con **Wit.ai**[3.6].

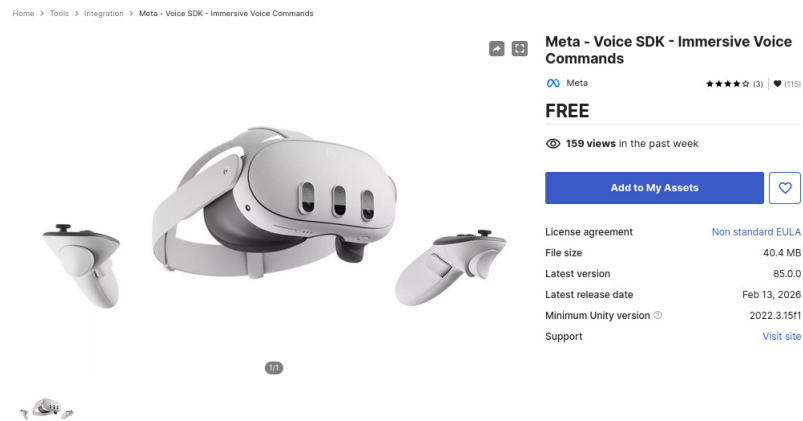


Figura 6: *Meta Voice SDK* nell'*Unity Asset Store*.

3.6 WIT.ai

È una piattaforma di *Natural Language Processing* per l'integrazione di funzionalità basate su comandi vocali. La piattaforma *WIT.ai*, insieme a *Meta Voice SDK* 3.5, viene utilizzata per l'integrazione dello *Speech-To-Text*.

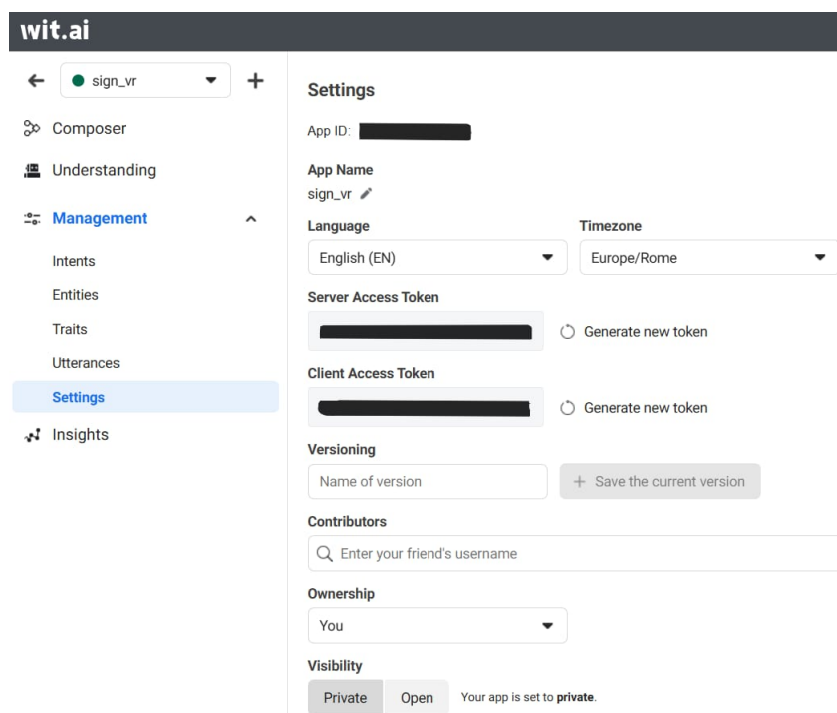


Figura 7: Pannello impostazioni della piattaforma *Wit.ai*.

3.7 LLaMA

Large Language Meta AI è una serie di modelli linguistici di grandi dimensioni⁶ *Open-Source* sviluppata da Meta Platforms, Inc[8]. L'impiego in **SignVR** è stato fondamentale per l'implementazione della traduzione in tempo reale da lingua inglese a *Gloss*: **SignVR** utilizza **LLaMA 3.3 70B Versatile**, un modello *Text-Only* e *Instruction-Tuned*⁷ che, attraverso l'inserimento di un *prompt*⁸, concatenato tra il dialogo ottenuto dalle funzionalità di *Speech-to-Text* e un *set* di istruzioni, fornisce in *output* il testo in notazione *Gloss*.

3.8 Piattaforma di Inferenza: Groq

È un servizio che offre capacità di inferenza⁹ ad alta velocità attraverso la tecnologia proprietaria *Language Processing Unit* (LPU)[6]. l'impiego di **Groq** nello sviluppo di **SignVR** è stato centrale in quanto garantisce tempi di risposta adatti ad una applicazione che opera in tempo reale.

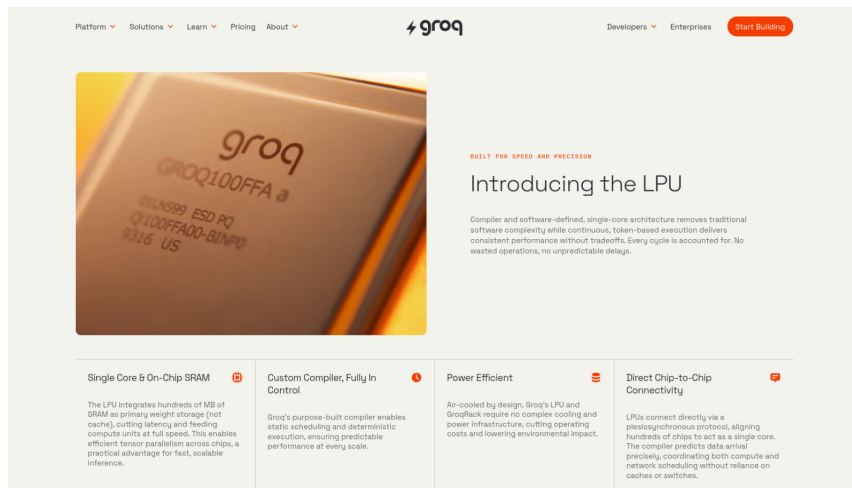


Figura 8: Articolo riguardante la tecnologia *LPU* dal sito ufficiale di **Groq**.

⁶i *Large Language Models* sono algoritmi di intelligenza artificiale capaci di generare un nuovo contenuto testuale a partire da grandi *dataset* preesistenti sfruttando tecnologie di *Deep Learning*.

⁷Nello sviluppo di modelli linguistici di grandi dimensioni, si riferisce allo sviluppo di un modello in modo da predisporlo alla comprensione e corretta esecuzione di istruzioni.

⁸nell'interazione con un modello di intelligenza artificiale generativa, un *prompt* è un *input* di istruzioni fornite dall'utente al fine di generare un contenuto.

⁹nell'ambito dell'intelligenza artificiale, è il processo che permette ad un modello di applicare le sue conoscenze pregresse a dei nuovi dati per trarre delle conclusioni.

3.9 Mixamo

È una piattaforma *Web-based* sviluppata da **Adobe** che fornisce strumenti avanzati di *Auto-Rigging* e una ampia libreria di modelli e animazioni[1]. L'utilizzo di **Mixamo** ha favorito lo sviluppo di **SignVR** grazie ai diversi modelli umani forniti dalla piattaforma.

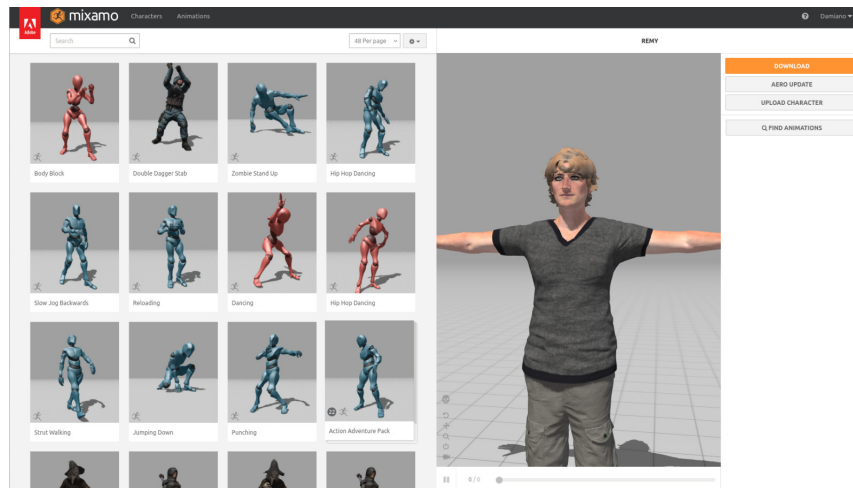


Figura 9: Intefracciia principale dell'applicazione **Mixamo**.

3.10 Affinity

È una *Suite* di strumenti creativi sviluppata da **Serif**[17]. Il *software* è stato utilizzato per la graficazione dei *Backplates* dell'interfaccia utente4.2 e la produzione di altri elementi grafici all'interno dell'applicazione.

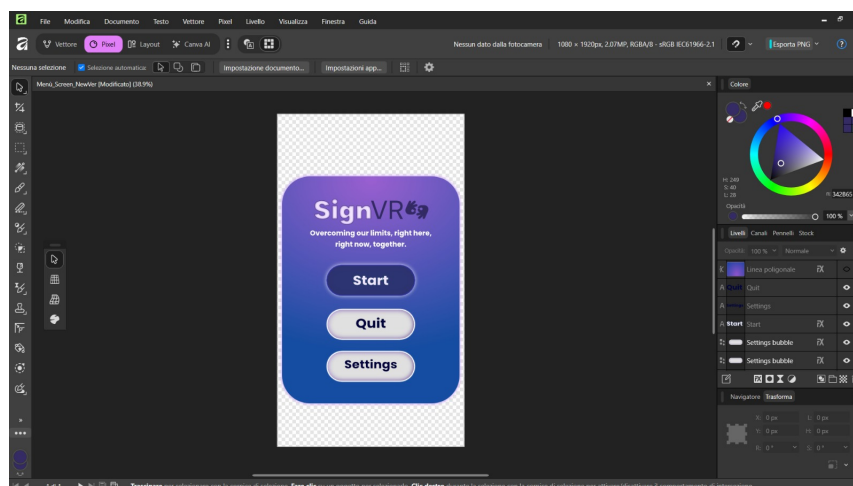


Figura 10: Schermata del *software* **Affinity**.

3.11 Blender

È un *software Open-Source* sviluppato dalla Blender Foundation[3]. Mira ad offrire una *Suite* di strumenti per modellazione ed animazione 3D, animazione 2D, *Compositing*, montaggio video e simulazioni fisiche. Il suo impiego nello sviluppo di **SignVR** è stato centrale per la normalizzazione e rielaborazione delle animazioni.

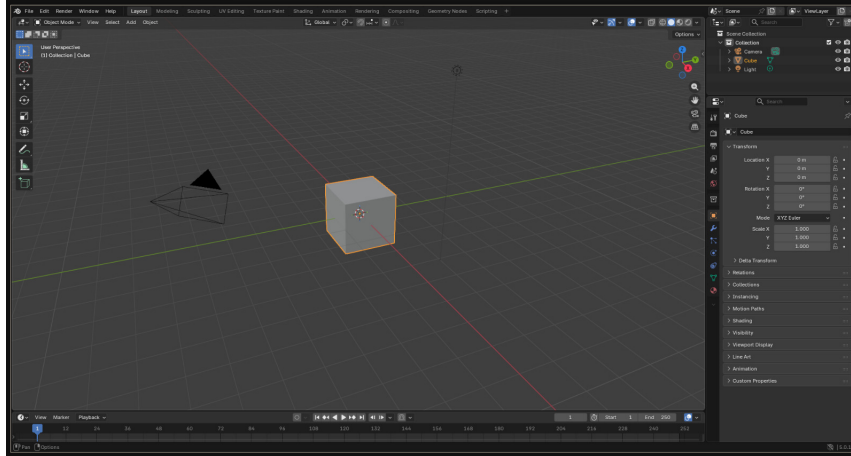


Figura 11: Scena di *default* del *software* **Blender**.

3.12 Animation Datasets

3.12.1 Provenienza delle animazioni

SignVR utilizza un set di animazioni ottenute tramite la normalizzazione di due *Datasets*:

- *3D-LEX v1.0: 3D Lexicons for American Sign Language and Sign Language of the Netherlands*[14]¹⁰
- *SLMocapArchive*[18]

3.12.2 Normalizzazione e rielaborazione

Alcune caratteristiche delle animazioni, come durata e *frame-rate* non erano favorevoli per l'utilizzo impiegato da **SignVR**, inoltre le *Key-Poses*¹¹ non erano allineate secondo uno *Standard* preciso. È stata effettuata una normalizzazione a 60 *frames* per secondo seguendo le seguenti linee guida:

¹⁰Lo studio, riguardante la cattura in 3D della lingua dei segni, fornisce un *Dataset* consultabile presso *Open Science Framework*

¹¹In animazione, pose principali che forniscono il contesto necessario ad interpretare il movimento che viene eseguito.

- Tutte le animazioni vengono riprodotte in 120 *frames*;
- **Frames 0-40:** il *rig* passa da posizione *idle*¹² al primo posizionamento utile alla riproduzione del segno;
- **Frames 40-80:** Il *rig* esegue il simbolo,
- **Frames 80-120** il *rig* passa dal gesto eseguito alla posizione *idle*.

È inoltre stata introdotta una animazione di *idle* e normalizzata la posizione iniziale e finale assunta dal *rig* in ogni animazione.

3.12.3 Limitazioni tecniche nell'implementazione delle animazioni

Data la diversa provenienza delle clip animate, il materiale raccolto presentava caratteristiche tecniche eterogenee, non essendo stato originariamente prodotto per un'applicazione specifica come **SignVR**. In particolare, i *rig* inclusi nei *dataset* risultavano privi di una struttura ossea facciale o di *shape keys* dedicate alla mimica. Come conseguenza diretta, l'implementazione delle *Non-Manual Expressions* non è stata possibile.

3.12.4 Implementazione limitata del vocabolario *ASL*

Vista la natura manuale del processo di normalizzazione delle animazioni, l'implementazione totale del vocabolario *ASL* all'interno di **SignVR** non è stata possibile. Per rendere il *software* funzionante in ogni scenario virtualmente possibile, viene riprodotto uno *spelling* quando incontra un termine non contenuto nel vocabolario. Il vocabolario di **SignVR** include, allo stato attuale, tutto l'alfabeto inglese ed un totale di 40 vocaboli.

4 Il software

4.1 Introduzione

L'obiettivo di **SignVR** è fornire un ambiente di supporto alla comunicazione per individui, oltre che una piattaforma di supporto allo sviluppo di competenze in *Gloss* e *American Sign Language*. Per ottenere il risultato desiderato è stato necessario seguire i seguenti punti chiave:

¹²In animazione, posizione di riposo assunta da un modello quando non sta performando nessuna azione.

- **Riduzione dei tempi di risposta e di attesa da parte dell'utente:** l'applicazione deve fornire informazioni all'utente con la maggior rapidità possibile, ciò non riguarda solo i tempi di risposta per la trascrizione in *Gloss* ma anche la celerità nella consegna della traduzione da parte dell'interprete, oltre che la durata della traduzione stessa;
- **Interfaccia intuitiva:** l'utente deve poter riconoscere intuitivamente il comportamento dei moduli che compongono l'interfaccia, essi devono essere auto-descrittivi, seguire una corretta gerarchia e vanno posizionati in un'ordine che rispetti il *workflow* del *software* stesso;
- **Accessibilità e supporto alla comunicazione:** il mantenimento del contatto visivo con l'interlocutore è fondamentale per una corretta comunicazione, l'interfaccia deve assecondare questa necessità: Ove il posizionamento *standard* dell'interfaccia non lo permetta, l'utente deve poter assumere il controllo, modificandone il posizionamento e comportamento al fine di venir incontro ad uno scenario specifico.



Figura 12: Schermata iniziale di **SignVR**.

4.2 L'interfaccia

SignVR permette la personalizzazione dell'esperienza utente offrendo un'interfaccia modulare ricomponibile attraverso *gestures*, opzioni riguardo la visibilità ed il posizionamento degli elementi a schermo e un catalogo di avatar selezionabili. Di seguito vengono analizzate nel dettaglio le tre componenti fondamentali dell'interfaccia.

4.2.1 *Speech-To-Text* e *Gloss Notation*

Il modulo è diviso in due *box* testuali. In ordine, questi contengono:

- La trascrizione del parlato, registrata dopo la pressione del tasto via *controller* o *gesture* da parte dell'utente fino al superamento della soglia di silenzio 4.6.1;
- La trascrizione da inglese a *Gloss*, operata da **LLaMA** 4.7.3.

Questa parte del programma viene divisa in due per poter permettere all'utente di oscurare o spostare i pannelli separatamente in base alle proprie esigenze: un utente che vuole imparare l'*American Sign Language* potrebbe preferire il *box Gloss* accanto all'interprete virtuale per confrontare prima la notazione con i gesti, allo stesso tempo un utente non udente potrebbe preferire l'ordine predefinito per consultare la trascrizione in *gloss* di ciò che gli è appena stato comunicato e averne una prima comprensione intuitiva.

4.2.2 Interprete Virtuale

Opposto al pannello di trascrizione del parlato vi è l'assistente virtuale, cui ruolo è fornire la traduzione dal formato di notazione *Gloss* alla lingua dei segni americana. L'avatar non è direttamente modificabile ma è possibile cambiarne le apparenze selezionando tra quattro possibili modelli. L'assistente, in assenza di *input Gloss* da tradurre, assumerà una posizione di *idle*, anche il suo posizionamento nell'interfaccia può essere modificato: per scenari dove la vista possa essere ostruita o ove l'utente voglia, ad esempio, posizionarlo accanto all'interlocutore; in scenari dove l'utilizzo dell'applicazione serva come strumento di apprendimento, un utente potrebbe voler concentrarsi sulla trascrizione in *Gloss* e voler oscurare l'interprete del tutto.



Figura 13: Eliza, interprete virtuale di *default* in **SignVR**



Figura 14: Giulio, interprete virtuale in **SignVR**

4.2.3 Impostazioni e personalizzazione dell'esperienza utente

È possibile accedere al pannello di impostazioni utilizzando il *controller Meta Quest* o attraverso *gestures*. Il pannello delle impostazioni permette di effettuare le seguenti operazioni:

- **Navigazione dell'applicazione:** accesso ai crediti, alla schermata di *Home*, tasto *Resume* per ritornare alla traduzione e chiusura dell'applicazione;
- **Display Avatar:** di norma attivo, permette di mostrare o nascondere l'interprete virtuale;
- **Display ASL Transcription:** di norma attivo, permette di mostrare o nascondere il pannello di trascrizione in *Gloss*;
- **Display English Transcription:** di norma attivo, permette di mostrare o nascondere il pannello di trascrizione del testo in lingua inglese;
- **Head Tracking Mode:** Comunica all'applicazione se l'interfaccia deve seguire lo sguardo dell'utente o rimanere fissa nella sua posizione di origine;

- **Avatar Selection:** Permette di scegliere quale *avatar* utilizzare come interprete virtuale;
- **Tutorial:** Permette di ricominciare il tutorial di **SignVR**.

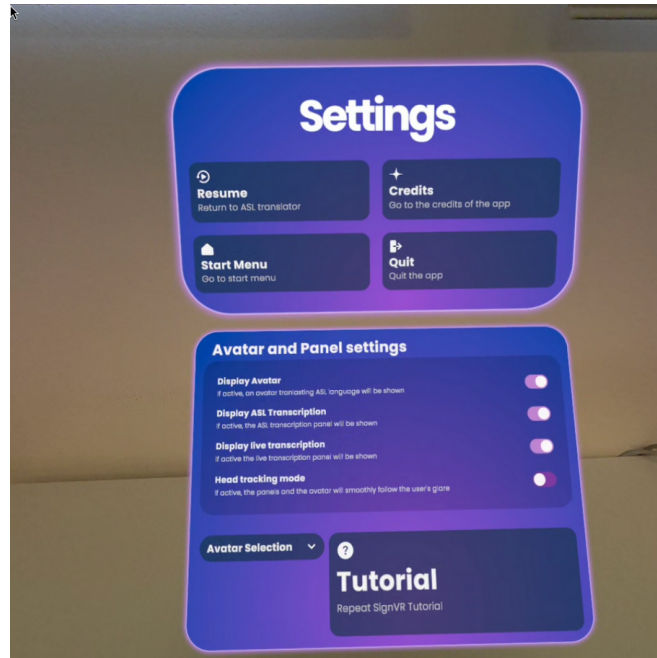


Figura 15: Pannello delle impostazioni in **SignVR**

4.2.4 Sezioni *Tutorial* e *Credits*

All'interno dell'applicazione vi sono anche un *Tutorial* interattivo, che spiega le funzioni principali del *software* e come potervi accedere, oltre che un pannello di *Credits* per ottenere maggiori informazioni sul lavoro degli sviluppatori.

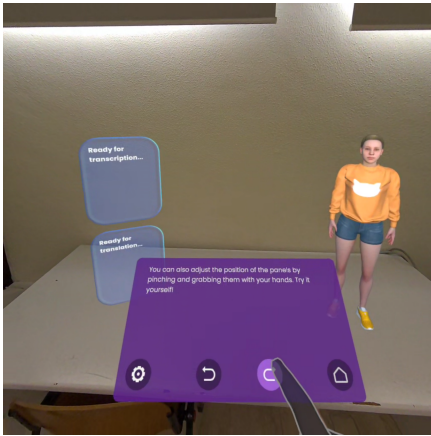


Figura 16: Scena del *tutorial* di **SignVR** riguardo l'interfaccia.

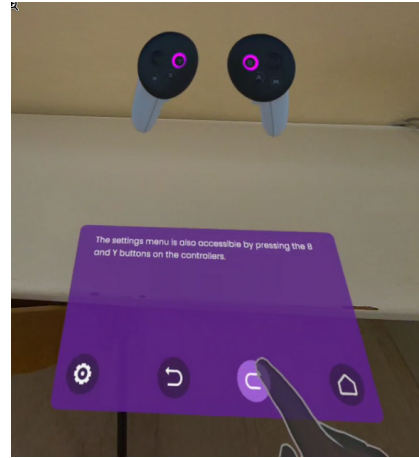


Figura 17: Scena del *tutorial* di **SignVR** sull'utilizzo dei *controller*.

4.2.5 Implementazione dell'interfaccia utente

L'interfaccia utente è stata costruita a partire dalle scene di esempio del *Meta XR Interaction SDK*, attraverso l'*Inspector* di **Unity**¹³ sono state modificate diverse proprietà, tra cui:

- Animazioni e suoni dei tasti;
- Dimensioni e *Aspect-Ratio* dei pannelli;
- *Backplates*¹⁴;
- Contenuto degli elementi testuali.

¹³In **Unity**, l'*Inspector* è il pannello che mostra tutte le proprietà di un oggetto all'interno della scena.

¹⁴Nell'ambito delle interfacce utenti, i *Backplates* sono contenitori, generalmente collocati dietro elementi dell'interfaccia stessa, che favoriscono la leggibilità e definiscono una gerarchia visiva.



Figura 18: Interfaccia principale di **SignVR**

4.3 Implementazione del pannello delle impostazioni e salvataggio delle *PlayerPrefs*

Il pannello delle impostazioni, anch'esso prodotto modificando scene d'esempio fornite dal *Meta XR Interaction SDK*, riflette le scelte dell'utente memorizzate attraverso *PlayerPrefs*¹⁵ e fornisce un'interfaccia per modificarle. Lo script *SettingsMenuLogic.cs* definisce una classe *SettingsMenuLogic* che possiede: 19

- **Attributi *Toggle*** per tutti i parametri cui visibilità può essere attivata o disattivata;
- **Metodi** per l'inizializzazione delle preferenze su impostazioni e scelta dell'*avatar* (*InitializeBooleanSettings* e *InitializeAvatarSelection*);

¹⁵In **Unity**, *PlayerPrefs* è una classe che conserva informazioni su scelte e impostazioni dell'utente.

- Metodi per il salvataggio delle singole *PlayerPrefs*;
- Metodo *Start* che esegue i metodi di inizializzazione.

```
public class SettingsMenuLogic : MonoBehaviour
{
    [Header("UI Toggles - General Settings")]
    public Toggle avatarToggle;
    public Toggle aslPanelToggle;
    public Toggle liveTranscriptToggle;
    public Toggle headTrackingToggle;

    [Header("Avatar Selection Group")]
    public ToggleGroup avatarSelection;

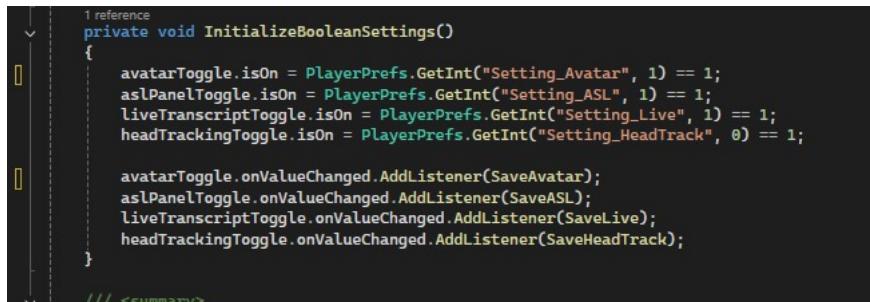
    ⓘ Unity Message | 0 references
    void Start()
    {
        InitializeBooleanSettings();
        InitializeAvatarSelection();
    }
}
```

Figura 19: Attributi e metodo start della classe *SettingsMenuLogic*.

La funzione *InitializeBooleanSettings* si occupa di inizializzare i *Toggle* sulla base delle *PlayerPrefs*: qual'ora non siano già presenti delle preferenze, la funzione le inizializza a dei valori di *default*, essa utilizza dei *Listener* per verificare se la preferenza viene successivamente modificata dall'utente.

La funzione *InitializeAvatarSelection* scorre la lista di tutti i possibili *avatar* selezionabili e li confronta con il *Toggle* selezionato dall'utente, poi carica l'*avatar* selezionato.

Le funzioni per il salvataggio delle *PlayerPrefs* sono mostrate in figura 22.



```

1 reference
private void InitializeBooleanSettings()
{
    avatarToggle.isOn = PlayerPrefs.GetInt("Setting_Avatar", 1) == 1;
    aslPanelToggle.isOn = PlayerPrefs.GetInt("Setting_AS_L", 1) == 1;
    liveTranscriptToggle.isOn = PlayerPrefs.GetInt("Setting_Live", 1) == 1;
    headTrackingToggle.isOn = PlayerPrefs.GetInt("Setting_HeadTrack", 0) == 1;

    avatarToggle.onValueChanged.AddListener(SaveAvatar);
    aslPanelToggle.onValueChanged.AddListener(SaveASL);
    liveTranscriptToggle.onValueChanged.AddListener(SaveLive);
    headTrackingToggle.onValueChanged.AddListener(SaveHeadTrack);
}

/// <summary>

```

Figura 20: Funzione *InitializeBooleanSettings* in *SettingsMenuLogic*.



```

private void InitializeAvatarSelection()
{
    int savedAvatarID = PlayerPrefs.GetInt("Setting_AvatarID", 0);
    Toggle[] dropdownToggles = avatarSelection.GetComponentInChildren<Toggle>().GetComponentsInChildren<Toggle>();

    for (int i = 0; i < dropdownToggles.Length; i++)
    {
        Toggle t = dropdownToggles[i];
        int index = i;

        if (i == savedAvatarID)
        {
            t.isOn = true;
        }

        t.onValueChanged.AddListener((isSelected) => {
            if (isSelected)
            {
                SaveAvatarID(index);
            }
        });
    }
}

```

Figura 21: Funzione *InitializeAvatarSelection* in *SettingsMenuLogic*.


```

1 reference
public void SaveAvatar(bool value)
{
    PlayerPrefs.SetInt("Setting_Avatar", value ? 1 : 0);
    PlayerPrefs.Save();
}

1 reference
public void SaveASL(bool value)
{
    PlayerPrefs.SetInt("Setting_AS�", value ? 1 : 0);
    PlayerPrefs.Save();
}

1 reference
public void SaveLive(bool value)
{
    PlayerPrefs.SetInt("Setting_Live", value ? 1 : 0);
    PlayerPrefs.Save();
}

1 reference
public void SaveHeadTrack(bool value)
{
    PlayerPrefs.SetInt("Setting_HeadTrack", value ? 1 : 0);
    PlayerPrefs.Save();
}

1 reference
public void SaveAvatarID(int id)
{
    PlayerPrefs.SetInt("Setting_AvatarID", id);
    PlayerPrefs.Save();
    Debug.Log($"[Settings] Avatar ID saved: {id}");
}

#endregion
}

```

Figura 22: Funzioni per il salvataggio delle *PlayerPrefs* in *SettingsMenuLogic*.

L'applicazione delle impostazioni nell'interfaccia avviene anche mediante lo *script* *SceneConfiguration.cs*, utilizzato per applicare le preferenze utente alla scena attiva. Lo *script* contiene una classe *SceneConfiguration* costituita da:

- **Attributi** *GameObject* e un attributo *LazyFollow* 4.9¹⁶;
- **Metodi** *Start()*, utilizzato all'avvio dello *script*, e *ApplySettings()*.

¹⁶*LazyFollow* è una classe utilizzata per permettere a **SignVR** di seguire lo sguardo dell'utente, viene affrontata al paragrafo 4.9.

```

{
    public GameObject avatarObject;
    public GameObject aslPanelObject;
    public GameObject livePanelObject;
    public LazyFollow headTrackingScript;
    void Start()
    {
        ApplySettings();
    }
}

```

Figura 23: Attributi e metodi della classe *SceneConfiguration*.

La funzione *ApplySettings()* modifica la proprietà *Object.SetActive()*, che comunica al motore se un *GameObject* deve essere mostrato all'utente; viene implementata come segue:

```

15 void ApplySettings()
16 {
17     // 1. Gestione Avatar
18     bool showAvatar = PlayerPrefs.GetInt("Setting_Avatar", 1) == 1;
19     if (avatarObject != null)
20         avatarObject.SetActive(showAvatar);
21
22     // 2. Gestione ASL Panel
23     bool showASL = PlayerPrefs.GetInt("Setting_AS�", 1) == 1;
24     if (aslPanelObject != null)
25         aslPanelObject.SetActive(showASL);
26
27     // 3. Gestione Live Transcription
28     bool showLive = PlayerPrefs.GetInt("Setting_Live", 1) == 1;
29     if (livePanelObject != null)
30         livePanelObject.SetActive(showLive);
31
32     // 4. Gestione Head Tracking
33     bool useHeadTracking = PlayerPrefs.GetInt("Setting_HeadTrack", 0) == 1;
34
35     if (headTrackingScript != null)
36         headTrackingScript.enabled = useHeadTracking;
37 }
38 }

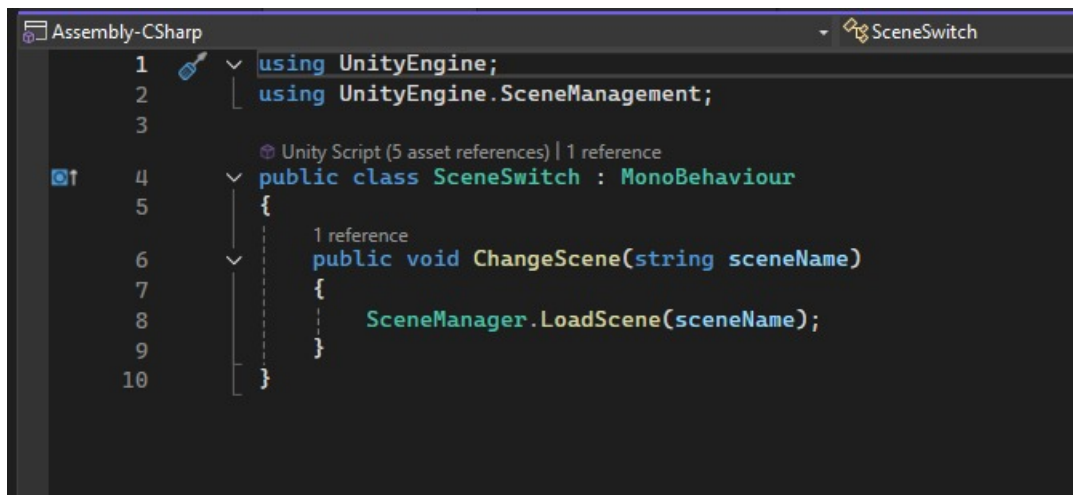
```

Figura 24: Funzione *ApplySettings()* in *SceneConfiguration*.

4.3.1 Gestione dell'interfaccia, *SceneManager* e chiusura dell'applicazione

Le diverse sezioni del programma sono organizzate all'interno di separate scene *Unity*¹⁷. Per il controllo e la gestione delle scene, **SignVR** utilizza l' API *SceneManager* messa a disposizione dal motore[19] e implementa diversi *scripts*:

- Il passaggio da una scena all'altra viene gestito dalla classe *SceneSwitch*, definita nello script *SceneSwitch.cs*:



```
1 using UnityEngine;
2 using UnityEngine.SceneManagement;
3
4 public class SceneSwitch : MonoBehaviour
5 {
6     public void ChangeScene(string sceneName)
7     {
8         SceneManager.LoadScene(sceneName);
9     }
10 }
```

Figura 25: *SceneSwitch* contiene una sola funzione che interagisce con *SceneManager* per cambiare scena.

- Il cambio di scena che permette l'apertura del pannello delle impostazioni è l'unico che non utilizza *SceneSwitch* direttamente:

¹⁷Una scena, in *Unity*, è un gruppo di *assets* e componenti raggruppati secondo uno scopo comune dallo sviluppatore.

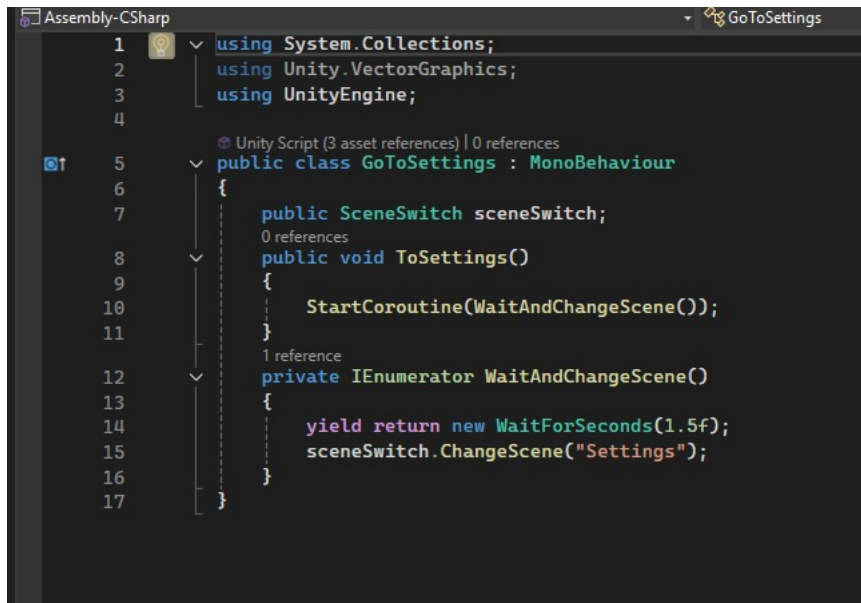


Figura 26: lo script *GoToSettings.cs* utilizza una *coroutine*, ovvero un metodo cui esecuzione può essere interrotta e successivamente ripresa in ogni momento[19].

Questo *script* viene utilizzato per aggiungere una latenza artificiale dalla richiesta effettuata dall'utente per accedere alle impostazioni fino al caricamento del pannello di impostazioni stesso: permette all'utente di visualizzare un messaggio a schermo in modo da non confonderlo durante l'utilizzo del *software*.

- La chiusura dell'applicazione avviene sempre utilizzando lo script *CloseApplication.cs*, implementato come segue:

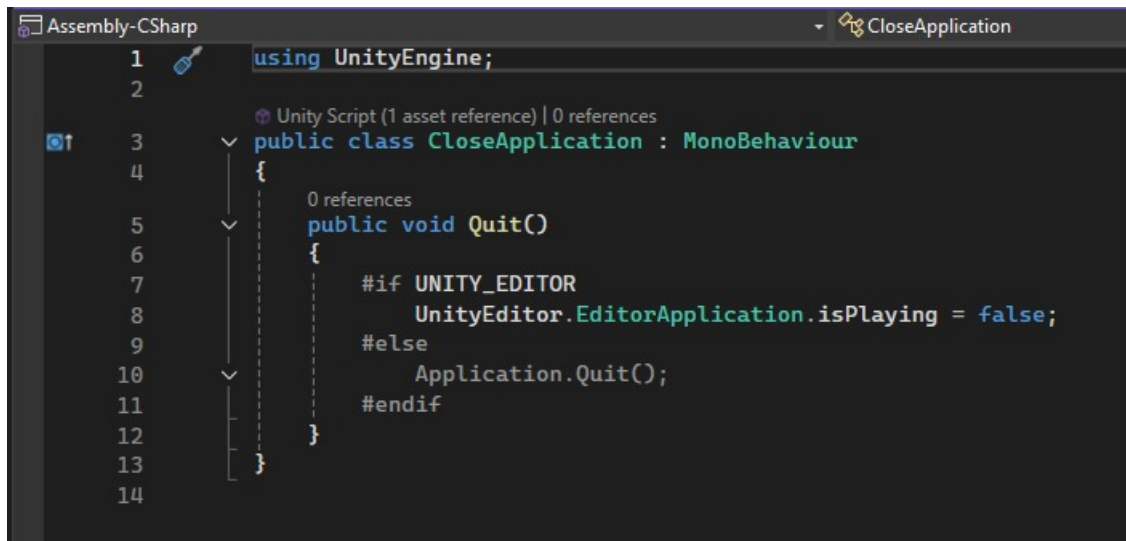


Figura 27: lo *script* definisce una classe *CloseApplication* con un solo metodo *Quit*.

4.4 Gestures

SignVR utilizza *gestures* per accedere a diverse funzionalità dell'applicazione, le *gestures* disponibili all'utente sono:

- ***Pinch***: utilizzato per selezionare e muovere elementi dell'interfaccia nello spazio;
- **Impostazioni**: di *default* posizionata sulla mano destra dell'utente, per accedere al pannello impostazioni;
- **Accesso allo *Speech-To-Text***: di *default* posizionata sulla mano sinistra dell'utente, permette l'attivazione del microfono per la *Speech-To-Text*.

4.4.1 Riconoscimento delle mani

Il riconoscimento delle mani avviene nativamente all'interno del *Meta Quest*. L'utilizzo del *Meta XR Interaction SDK* permette di utilizzare tale riconoscimento al fine di implementare *gestures* all'interno del *software*.

4.4.2 Implementazione delle gestures

Le *gestures* vengono importate dalle scene d'esempio del *Meta XR Interaction SDK* per essere successivamente rielaborate:

- Vengono modificati gli *scripts* chiamati durante l'utilizzo delle *gestures*;

- viene modificata la resa visiva del bottone per lo *Speech-To-Text*;
- viene modificato il comportamento della *gesture* impostazioni¹⁸.

4.5 Implementazione al supporto per controller Meta Quest

Il supporto per *controller Meta Quest* viene integrato attraverso **Building Blocks**. I controlli vengono re-impostati per seguire le esigenze del *software*, in particolare:

- **Bottoni X, A:** permettono di attivare e disattivare lo *Speech-To-Text*;
- **Bottoni Y, B:** permettono di accedere alle impostazioni;
- **Primary & Secondary Index Trigger:** permettono di eseguire il *pinch* degli elementi dell'interfaccia.

4.6 Implementazione dello *Speech-To-Text*

La funzione di *Speech-To-Text* permette la ripresa e successiva trascrizione del parlato inglese. Viene implementata utilizzando *Meta Voice SDK* ed il servizio **Wit.ai**, in particolare:

- Viene importato lo *script* che definisce la classe *AppDictationExperience*¹⁹;
- vengono apportate modifiche riguardo i tempi di ripresa dell'audio del parlato;
- viene integrato il sistema di **Wit.ai** per ottenere il testo della trascrizione.

¹⁸Originariamente chiamata *scissor* nel *Meta XR Interaction SDK*.

¹⁹Classe definita nel *Meta Voice SDK*, si occupa di gestire la dettatura.

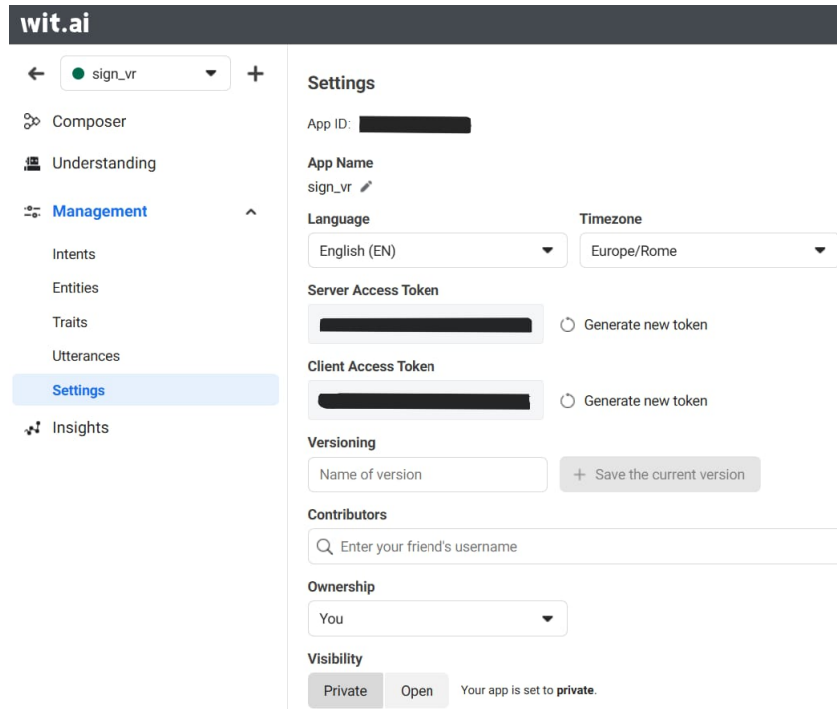


Figura 28: Interfaccia di *Wit.ai*.

4.6.1 Periodo di ascolto e soglia di silenzio

La ripresa del parlato in lingua inglese è condizionata dal tempo di esecuzione e dal livello di pressione sonora rilevato dal microfono collegato o integrato al visore.

Meta Voice SDK definisce la classe *AppDictationExperience*, che permette la modifica, mediante l'*inspector* di **Unity**, dei parametri che controllano il tempo di ripresa. L'applicazione è stata testata in ambienti con basso e alto profilo di rumore ambientale, utilizzando il microfono integrato al *Meta Quest* per il parlato dell'utente che indossa il visore²⁰.

4.7 Implementazione della trascrizione in *Gloss*

La trascrizione dall'inglese all'*American Sign Language* viene gestita mediante **LLaMA**. Vengono implementati i seguenti *scripts*:

- ***VoiceDictationManager.cs***: Si occupa di ricevere il testo in lingua inglese, mostrarlo a schermo e mandarlo al *Large Language Model* per essere trascritto;

²⁰La *feature* di *Noise Cancelling* che *Meta Quest* applica al microfono integrato rende impossibile la ripresa del parlato da parte di un interlocutore terzo, richiedendo l'utilizzo di un microfono esterno.

- *LlmTranslationHandler.cs*: fornisce un *prompt* al *Large Language Model*, gestisce le richieste e riceve la risposta contenente il testo in *Gloss*.

4.7.1 Ricezione del testo e *VoiceDictationManager*

Lo *script* definisce una classe *VoiceDictationManager* con diversi attributi per:

- Controllare le icone mostrate a schermo durante l'utilizzo dello *Speech-To-Text*;
- Modificare il testo della trascrizione in lingua inglese;
- Integrare il *Large Language Model*;
- Integrare la classe *AppDictationExperience*.

```
public class VoiceDictationManager : MonoBehaviour
{
    [Header("Voice Engine")]
    [SerializeField] private AppDictationExperience dictationExperience;

    [Header("UI Interface")]
    [SerializeField] private TMP_Text transcriptionText;
    [SerializeField] private Image microphoneStatusImage;

    [Header("LLM Integration")]
    [SerializeField] private LlmTranslationHandler llmHandler;

    private bool _isRecording = false;
}
```

Figura 29: Attributi della classe *VoiceDictationManager*.

La classe *VoiceDictationManager* implementa metodi per attivare o disattivare la dettatura aggiornando conseguentemente l'UI, come visto in figura 30, e per la gestione della trascrizione parziale e completa del parlato ²¹, con successiva modifica del testo dell'interfaccia e invio al modello di linguaggio, presenti in figura 31.

²¹Con trascrizione parziale si intende la trascrizione che avviene quando l'interlocutore sta ancora parlando e la ripresa è attiva, la trascrizione è considerata completa quando la dettatura è stata interrotta.


```

private void StartRecording()
{
    UpdateUIState(Color.green);
    _isRecording = true;
    if (dictationExperience == null) return;

    dictationExperience.Activate();
}

private void StopRecording()
{
    UpdateUIState(Color.white);
    _isRecording = false;
    if (dictationExperience == null) return;

    dictationExperience.Deactivate();
}

```

Figura 30: Funzioni che controllano la *AppDictationExperience* e che modificano il colore della UI in *VoiceDictationManager.cs*.

```

private void OnPartialTranscriptionReceived(string text)
{
    if (transcriptionText != null)
        transcriptionText.text = text + "...";
}

private void OnFullTranscriptionReceived(string text)
{
    if (transcriptionText != null)
        transcriptionText.text = text;

    Debug.Log($"Testo ricevuto (Voce o Debug): {text}");

    if (llmHandler != null && !string.IsNullOrWhiteSpace(text))
    {
        llmHandler.SendTranslationRequest(text);
    }
}

```

Figura 31: Funzioni che controllano la trascrizione totale e parziale, con conseguente aggiornamento del testo nell'interfaccia utente.

4.7.2 Definizione del *Prompt* per la trascrizione in *Gloss*

Il testo ottenuto dalla trascrizione completa, gestita dal *VoiceDictationManager* e dalla *AppDictationExperience*, viene mostrato all'utente nel pannello apposito e inviato a **LLaMA** per la rielaborazione in *Gloss*. In **SignVR**, il modello di linguaggio opera seguendo un *prompt* che lo istruisce sulla procedura da seguire:

- Viene fornito contesto sul ruolo del modello;
- vengono fornite le regole di *glossing*;

- vengono forniti esempi sfruttando il *Few-Shot Learning* ²²;
- viene fornita una *Task* che il modello deve seguire.

Il *prompt* viene concatenato alla trascrizione ed inviato al modello di linguaggio, mentre la risposta del modello viene mostrata all'utente attraverso l'interfaccia.

Segue il *prompt* utilizzato:²³

²²Il *Few-Shot Learning* è un approccio utilizzato in *Machine Learning*, permette la generalizzazione di un concetto sfruttando le conoscenze pregresse del modello ed una serie di esempi limitata.

²³La versione presente in **SignVR** include diversi esempi, omessi in questo documento per brevità.

You are an expert American Sign Language (ASL) translator.

Your task is to translate standard English text into ASL Gloss notation.

GLOSSING RULES:

1. FORMAT: Output ONLY the translated glosses in UPPERCASE.
No explanations, no punctuation, no question marks.
2. GRAMMAR STRUCTURE: Use the order:
TIME + OBJECT + SUBJECT + VERB + QUESTION-WORD/NEGATION.
3. OMISSIONS: Remove all articles (a, an, the),
remove all 'to be' verbs (am, is, are, was, were),
and remove prepositions (to, of) unless necessary for direction.
4. VERBS & NOUNS: Use the root form of words. Remove '-ing', '-ed',
and plural 's'. (e.g., 'cats' -> 'CAT', 'walking' -> 'WALK').
5. ADJECTIVES: Place adjectives AFTER the noun (e.g., 'Red car' -> 'CAR RED').
6. NEGATION: Place 'NOT' or 'NONE' at the END of the sentence
or immediately before the verb.
7. QUESTIONS: Place WH-words (WHO, WHAT, WHERE, WHEN, WHY, HOW) at the VERY END.
8. PRONOUNS (CRITICAL): Preserve the exact point of view.
If input is 'YOU', output is 'YOU'. Do NOT answer the question.
9. GREETINGS: Do not remove greetings such as hi or hello.

EXAMPLES (Few-Shot Learning):

Input: I am going to the store tomorrow.

Output: TOMORROW STORE I GO

...

Input: I have two dogs.

Output: DOG TWO I HAVE

TASK:

Translate the following English text to ASL Gloss(...)

4.7.3 Gestione dell'LLM attraverso *LlmTranslationHandler*

Lo script *LlmTranslationHandler.cs* definisce la classe *LlmTranslationHandler*, che gestisce le richieste poste a **LLaMA** e le rispettive risposte. La classe possiede attributi per la gestione dell'LLM, gestione delle animazioni e testo della notazione in *Gloss*.

```
public class LlmTranslationHandler : MonoBehaviour
{
    [Header("LLM Integration")]
    [SerializeField] private LlmAgent llmAgent;

    [Header("Animator Handler")]
    [SerializeField] private AvatarAnimatorHandler animatorHandler;

    [Header("UI Display")]
    [SerializeField] private TMP_Text aslGlossText;
}
```

Figura 32: Attributi della classe *LlmTranslationHandler*. Tra gli attributi della classe vi è anche il *prompt*, affrontato al paragrafo 4.7.2, omissso per brevità.

La classe implementa i metodi *SendTranslationRequest()*, per inviare il testo al modello di linguaggio, e *HandleLlmResponse()*, che riformatta il testo, lo mostra sullo schermo e chiama l'*AnimatorHandler* per eseguire l'animazione.

```
public async void SendTranslationRequest(string englishText)
{
    if (llmAgent == null)
    {
        Debug.LogError("LlmTranslationHandler: LlmAgent non assegnato.");
        return;
    }

    string fullPrompt = (SYSTEM_PROMPT + englishText).Replace("\n", "");

    try
    {
        await llmAgent.SendTextOnlyAsync(fullPrompt);
    }
    catch (System.Exception e)
    {
        Debug.LogError($"Errore durante la richiesta LLM: {e.Message}");
        if (aslGlossText != null) aslGlossText.text = "Error translating.";
    }
}

private void HandleLlmResponse(string response)
{
    // Pulizia base della risposta (rimozione spazi extra)
    response = response.Trim();

    if (aslGlossText != null)
    {
        aslGlossText.text = response;
    }

    if (animatorHandler != null)
    {
        animatorHandler.ProcessAndAnimate(response);
    }
}
```

Figura 33: Metodi implementati nella classe *LlmTranslationHandler*.

4.8 Animazioni e interpreti virtuali

4.8.1 Standardizzazione e interpolazione delle animazioni

Le animazioni utilizzate da **SignVR** sono frutto della normalizzazione dei *dataset* utilizzati²⁴, che segue le regole affrontate al paragrafo 3.12.2. La normalizzazione delle animazioni rende prevedibile la durata, nonché il posizionamento dei movimenti chiave su tutte le parole del vocabolario. Al verificarsi di una traduzione contenente due o più parole, si avrebbero un totale di 80 *frames* di riposizionamento da *key-pose* ad *idle* e viceversa, a fronte di 40 *frames* di animazione effettiva, introducendo informazioni non necessarie all'utente e rallentando la traduzione, oltre che forzando l'interprete virtuale ad assumere un comportamento innaturale. Per ovviare a questo problema, le animazioni vengono tagliate ed interpolate: gli ultimi 40 *frames* della prima animazione e i primi 40 della seconda vengono tagliati, scartando 80 *frames* superflui, viene poi applicato il metodo *CrossFade()* per transitare da un'animazione all'altra. ciò permette di rimuovere informazioni non fondamentali, mantenere naturalezza nelle animazioni dell'interprete e utilizzare le stesse animazioni indipendentemente dal loro posizionamento all'interno della frase.

4.8.2 *AnimationHandler* come coordinatore delle animazioni

La gestione delle animazioni viene affidata all'*AnimationHandler*, classe definita nello *script AnimationHandler.cs*. La classe possiede un attributo *Animator* per il controllo delle animazioni all'interno del motore **Unity**, attributi riguardo il *frame-rate* e la durata delle animazioni ed una Coda *First-In First-Out*²⁵ per gestire la coda delle animazioni.

```
[Header("Avatar & Timing")]
[SerializeField] private Animator avatarAnimator;

[Tooltip("Frame rate delle clip di animazione")]
public float frameRate = 60f;

[Tooltip("Durata totale in frame delle clip")]
public float totalFramesClip = 120f;

public float crossFadeTime = 0.1f;

// Coda FIFO per gestire sequenze di frasi senza interruzioni
private Queue<string> animationQueue = new Queue<string>();
private bool isAnimating = false;
```

Figura 34: Attributi della classe *AnimationHandler*.

²⁴Si noti che la normalizzazione è stata effettuata su un numero limitato di animazioni e non su tutte quelle disponibili dai *datasets* 3.12.3.

²⁵Le code *FIFO* sono strutture dati dove l'ordine di rimozione degli elementi è condizionato all'ordine in cui sono essi vengono aggiunti: il primo elemento aggiunto alla lista sarà il primo ad uscire.

AnimationHandler possiede diversi metodi per la corretta gestione della animazioni: *ProcessQueueRoutine()* si occupa di scorrere la coda una stringa per volta, attendendo il completamento di ogni animazione:

```
// Processa la coda delle frasi una alla volta, attendendo il completamento di ciascuna
private IEnumerator ProcessQueueRoutine()
{
    isAnimating = true;

    while (animationQueue.Count > 0)
    {
        string nextPhrase = animationQueue.Dequeue();

        yield return StartCoroutine(AnimatePhraseRoutine(nextPhrase));
    }

    // Ritorno allo stato Idle alla fine della sequenza completa di frasi
    if (avatarAnimator != null)
        avatarAnimator.CrossFade("Idle", 0.2f);

    isAnimating = false;
}
```

Figura 35: Implementazione della funzione *ProcessQueueRoutine()*.

La funzione *ProcessAndAnimate()* si occupa di aggiungere nuove stringhe alla coda; se non ci sono animazioni in corso si occupa anche di chiamare *ProcessQueueRoutine()*.

```
// Accoda la frase e avvia la coroutine di
// gestione coda se non è già in esecuzione
public void ProcessAndAnimate(string phrase)
{
    animationQueue.Enqueue(phrase);

    if (!isAnimating)
    {
        StartCoroutine(ProcessQueueRoutine());
    }
}
```

Figura 36: Implementazione della funzione *ProcessAndAnimate()*.

La gestione della singola stringa di testo viene affidata al metodo *AnimatePhraseRoutine()*: La stringa viene divisa in parole, ognuna elemento di un *array*, verifica se sono presenti nel vocabolario e comunica se queste sono la prima o l'ultima ad essere riprodotte.

```

private IEnumerator AnimatePhraseRoutine(string phrase)
{
    phrase = phrase.ToUpper();
    string[] words = phrase.Split(' ', System.StringSplitOptions.RemoveEmptyEntries);

    for (int i = 0; i < words.Length; i++)
    {
        string word = words[i];

        bool isFirstWord = (i == 0);
        bool isLastWord = (i == words.Length - 1);

        // Hash precalcolato per verifica performante nel controller
        int wordHash = Animator.StringToHash(word);

        // Verifica se esiste un'animazione specifica per l'intera parola
        bool fullWordExists = avatarAnimator.HasState(0, wordHash);

        if (fullWordExists)
        {
            yield return StartCoroutine(PlayWordAnimation(word, isFirstWord, isLastWord));
        }
        else
        {
            // Fallback: esegue lo spelling lettera per lettera se manca la parola intera
            yield return StartCoroutine(PlayWordSpelling(word));
        }
    }
}

```

Figura 37: Implementazione della funzione *AnimatePhraseRoutine()*.

Le funzioni *PlayWordAnimation()* e *PlayWordSpelling()* vengono invocate sulla base del controllo eseguito sulle singole parole dell'*array*: le parole presenti nel vocabolario vengono riprodotte:

- Per intero se la parola è sia la prima che l'ultima;
- tagliando gli ultimi 30 *frames* se la parola è la prima;
- tagliando i primi 30 *frames* se la parola è l'ultima;
- tagliando i primi e gli ultimi 30 *frames* se la parola è in una posizione generica.

La riproduzione delle parole in sequenza avviene mediante la funzione *CrossFade()*, definita nella classe *Animator*; La funzione *PlayWordSpelling()* segue lo stesso principio ma converte la parola ricevuta in un *array* di caratteri.

```

private IEnumerator PlayWordAnimation(string stateName, bool isFirst, bool isLast)
{
    float startFrame = 0;
    float durationFrames = 0;

    // Calcolo start/duration per gestire Entry (inizio), Hold (centro) ed Exit (fine)
    // ritagliando le porzioni di clip non necessarie in base alla posizione
    if (isFirst && isLast) // Frase di una sola parola
    {
        startFrame = 0;
        durationFrames = totalFramesClip;
    }
    else if (isFirst) // Prima parola della frase (Entry + Hold)
    {
        startFrame = 0;
        durationFrames = 90; // Taglia l'uscita
    }
    else if (isLast) // Ultima parola della frase (Hold + Exit)
    {
        startFrame = 30; // Salta l'entrata
        durationFrames = totalFramesClip - 30;
    }
    else // Parola centrale (Solo Hold)
    {
        startFrame = 30; // Salta entrata
        durationFrames = 60; // Esegui solo parte centrale, taglia uscita
    }

    // Offset normalizzato (0-1) richiesto dal CrossFade per partire dal frame corretto
    float normalizedStartOffset = startFrame / totalFramesClip;

    avatarAnimator.CrossFade(stateName, crossFadeTime, 0, normalizedStartOffset);

    // Calcola il tempo di attesa basato sul taglio effettuato
    float waitTime = durationFrames / frameRate;

    yield return new WaitForSeconds(waitTime);
}

```

Figura 38: Implementazione della funzione *PlayWordAnimation()*.

```

private IEnumerator PlayWordSpelling(string word)
{
    char[] letters = word.ToCharArray();

    for (int i = 0; i < letters.Length; i++)
    {
        char currentLetter = letters[i];

        if (char.IsLetter(currentLetter))
        {
            string letterStateName = currentLetter.ToString();
            float startFrame = 0;
            float durationFrames = 0;

            bool isFirst = (i == 0);
            bool isLast = (i == letters.Length - 1);

            // Timing specifici per le lettere (diversi dalle parole intere)
            if (isFirst && isLast)
            {
                startFrame = 0;
                durationFrames = totalFramesClip;
            }
            else if (isFirst)
            {
                startFrame = 0;
                durationFrames = 80;
            }
            else if (isLast)
            {
                startFrame = 40;
                durationFrames = totalFramesClip - 40;
            }
            else
            {
                startFrame = 40;
                durationFrames = 40;
            }

            float normalizedStartOffset = startFrame / totalFramesClip;

            avatarAnimator.CrossFade(letterStateName, crossFadeTime, 0, normalizedStartOffset);

            float waitTime = durationFrames / frameRate;
            yield return new WaitForSeconds(waitTime);
        }
    }
}

```

Figura 39: Implementazione della funzione *PlayWordSpelling()*.

4.9 *Lazy-Follow* e movimento dell'interfaccia

Il comportamento dell'interfaccia rispetto al movimento del capo dell'utente viene gestito dallo *script LazyFollow.cs*, con l'obiettivo di modificare la posizione dei moduli seguendo il movimento del visore. Questa funzionalità può essere disattivata

dall'utente^{4.2.3} lasciando che la gestione degli elementi a schermo venga gestita dal sistema di *boundaries* e confini del *Meta Quest*.

4.9.1 Attributi dello *script* e definizione del comportamento

Viene definito un attributo *Dead Zone Angle* che rappresenta l'angolo massimo di movimento del capo dell'utente rispetto l'interfaccia prima che lo *script* entri in azione. Vengono anche definiti *Dead Zone Distance* e *Follow Speed Multiplier*, variabili che controllano rispettivamente la distanza massima dell'utente dall'interfaccia e la velocità con cui la stessa raggiunge il punto calcolato dallo *script*.

```
{
    [Header("Target & Settings")]
    public Transform target; // Assegna la Main Camera (CenterEyeAnchor)

    [Header("Deadzone Settings (LAZY FOLLOW)")]
    [Tooltip("L'angolo massimo prima che il menu inizi a seguire la testa.")]
    [Range(0f, 90f)]
    public float deadzoneAngle = 25.0f;

    [Tooltip("La distanza massima prima che il menu ti segua (utile se cammini via).")]
    public float deadzoneDistance = 0.5f;

    [Tooltip("Quanto velocemente il menu scatta quando esce dalla deadzone.")]
    public float followSpeedMultiplier = 1.0f; // 1 = velocità normale, >1 più reattivo

    [Header("Position Settings")]
    public Vector3 positionOffset = new Vector3(0, 0, 1.0f);
    public float posSmoothTime = 0.3f;

    [Space(5)]
    public bool lockPosX = false;
    public bool lockPosY = false;
    public bool lockPosZ = false;

    [Header("Rotation Settings")]
    public Vector3 rotationOffset = Vector3.zero;
    public float rotSpeed = 5.0f;

    [Space(5)]
    public bool rotateOnlyWhileMoving = true; // Se VERO, ruota solo quando si riposiziona (più stabile)
    public bool lockRotX = true;
    public bool lockRotY = false;
    public bool lockRotZ = true;

    // Variabili private per la logica
    private Vector3 velocity = Vector3.zero;
    private bool isFollowing = false;
    private Vector3 targetDestination; // La posizione dove vogliamo andare
}
```

Figura 40: Attributi della classe *LazyFollow*.

Vengono definiti altri attributi per il posizionamento dell'interfaccia, per il controllo della rotazione e per la logica della classe.

4.9.2 Avvio dello *script* e *ForceUpdatePosition()*

Il metodo *ForceUpdatePosition()* pone la posizione del *GameObject* a cui è applicato lo *script* ad un *offset* rispetto la telecamera²⁶; lo *script* è sempre applicato al *GameObject* rappresentante l'interfaccia.

²⁶Con camera si fa riferimento al *Camera Rig*, implementato mediante **Meta Building Blocks**.

```

public void ForceUpdatePosition()
{
    if(target == null) return;
    transform.position = target.TransformPoint(positionOffset);

    if(lockPosY)
    {
        Vector3 p = transform.position;
        p.y = target.position.y + positionOffset.y;
        transform.position = p;
    }

    HandleRotation();
    // Azzerà velocità per evitare "scivolamenti" dopo il teletrasporto
    velocity = Vector3.zero;
    isFollowing = false;
}

```

Figura 41: Implementazione del metodo *ForceUpdatePosition()* nella classe *Lazy-Follow*.

ForceUpdatePosition() viene sempre eseguito all'interno del metodo *Start()*.

4.9.3 Funzione di *LateUpdate()*, controllo della posizione e movimento dell'interfaccia

Viene ridefinita la funzione *LateUpdate()* al fine di:

- Calcolare la distanza ideale tra *camera* e pannelli;
- calcolare l'eventuale trasformazione da applicare al *GameObject*, tenendo in considerazione eventuali blocchi sulle assi;
- abilitare e disabilitare il *Following*, che indica quando è necessario applicare una trasformazione;
- applicare trasformazioni di posizione e rotazione al *GameObject*.

```

void LateUpdate()
{
    if (target == null) return;

    // 1. CALCOLA LA POSIZIONE IDEALE
    Vector3 idealPos = target.TransformPoint(positionOffset);

    float destX = lockPosX ? transform.position.x : idealPos.x; //LockPos TRUE --> transform.pos...
    float destY = lockPosY ? transform.position.y : idealPos.y; //
    float destZ = lockPosZ ? transform.position.z : idealPos.z; //LockPos FALSE -> idealPos...

    Vector3 finalWantedPos = new Vector3(destX, destY, destZ);

    // 2. CONTROLLO DEADZONE (Logica Trigger)
    float distance = Vector3.Distance(transform.position, finalWantedPos);

    // Calcolo angolo: angolo tra "dove guardo" (target.forward) e "dov'è il menu" (dirToMenu)
    Vector3 dirToMenu = (transform.position - target.position).normalized;
    float angle = Vector3.Angle(target.forward, dirToMenu);

    // SE non stiamo muovendo, controlliamo se dobbiamo iniziare
    if (!isFollowing)
    {
        // Trigger attivazione: Angolo eccessivo O Distanza eccessiva
        if (angle > deadzoneAngle || distance > deadzoneDistance)
        {
            isFollowing = true;
        }
    }
}

```

Figura 42: Calcolo della posizione ideale e della posizione finale, verifica delle condizioni necessarie per abilitare il *Following* in *LateUpdate()*.

```

// 3. MOVIMENTO (Se attivo)
if (isFollowing)
{
    // Aggiorniamo la destinazione continuamente mentre seguiamo
    targetDestination = finalWantedPos;

    // Usiamo SmoothDamp. Nota: Moltiplichiamo il tempo per ridurre la latenza se necessario
    transform.position = Vector3.SmoothDamp(transform.position, targetDestination, ref velocity,
    posSmoothTime / followSpeedMultiplier);

    // Condizione di arresto: Se siamo molto vicini alla destinazione e allineati
    // Usiamo una soglia piccola (es. 0.01 unità e 1 grado)
    if (Vector3.Distance(transform.position, targetDestination) < 0.01f && angle < 1.0f)
    {
        isFollowing = false;
    }
}

// 4. ROTAZIONE
// Spesso è meglio ruotare solo quando ci stiamo muovendo per evitare micro-jittering mentre leggiamo
if (isFollowing || !rotateOnlyWhileMoving)
{
    HandleRotation();
}
}

```

Figura 43: Applicazione della trasformazione e rotazione in *LateUpdate()*.

4.9.4 Gestione della rotazione tramite *HandleRotation()*

Il metodo, definito nello *script LazyFollow.cs*, viene utilizzato per gestire l'applicazione della rotazione. Vengono identificati gli angoli di rotazione che devono essere raggiunti sulla base della presenza o meno di un blocco imposto, viene successivamente applicata la rotazione.

```

void HandleRotation()
{
    Vector3 targetEuler = target.eulerAngles;

    float finalRotX = lockRotX ? rotationOffset.x : targetEuler.x + rotationOffset.x;
    float finalRotY = lockRotY ? rotationOffset.y : targetEuler.y + rotationOffset.y;
    float finalRotZ = lockRotZ ? rotationOffset.z : targetEuler.z + rotationOffset.z;

    Quaternion desiredRotation = Quaternion.Euler(finalRotX, finalRotY, finalRotZ);

    transform.rotation = Quaternion.Slerp(transform.rotation,
    desiredRotation, Time.deltaTime * rotSpeed);
}

```

Figura 44: Implementazione del metodo *HandleRotation()* in *LazyFollow*.

Lo *script* rappresenta la rotazione mediante quaternioni e utilizza la funzione *Quaternion.Slerp()* per applicare la transizione tra la posizione precedente e quella desiderata.

5 Note conclusive

Il *software* è stato sviluppato da Davide Leotta, Elena Giunta e Damiano Rinaldi, studenti del CdL triennale in Informatica presso l'Università degli Studi di Catania. Per quanto il gruppo di lavoro abbia sviluppato **SignVR** in sinergia, vi è stata una suddivisione degli incarichi e definizione dei ruoli per velocizzarne lo sviluppo.

La produzione degli *assets* grafici, *design* e sviluppo del comparto *UI/UX*²⁷ vengono implementate da **Elena Giunta**.

La gestione delle animazioni, integrazione dell'*LLM* e l'implementazione della funzionalità di *Lazy Follow* sono state realizzate da **Davide Leotta**.

La prototipazione, il *Wireframing* e l'implementazione di alcune funzionalità *UI* sono state attuate da **Damiano Rinaldi**.

²⁷ *User Interface* e *User Interaction*.

Riferimenti bibliografici

- [1] Adobe Inc. *Mixamo: Animate Digital Characters for Games, Shows, and More*. Accessed: 2026-02-12. Adobe Systems. 2026. URL: <https://www.mixamo.com/>.
- [2] Santiago Berrezueta-Guzman e Stefan Wagner. «Choosing the Right Engine in the Virtual Reality Landscape: A Comparative Study». In: *arXiv preprint* (2025). Accessed: 2026-02-12. URL: <https://arxiv.org/html/2508.13116v1>.
- [3] Blender Online Community. *Blender - a 3D modelling and rendering package*. Accessed: 2026-02-13. Blender Foundation. Amsterdam, 2026. URL: <https://www.blender.org>.
- [4] VR-compare. *Meta Quest 3 Full Specifications*. <https://vr-compare.com/headset/metaquest3>. Accessed: 2026-02-12.
- [5] Ethnologue. «American Sign Language». In: *Ethnologue* (2022). Accessed: 2026-02-11. URL: <https://www.ethnologue.com/language/ase/>.
- [6] Groq Inc. *Groq LPU Inference Engine: Real-time AI Acceleration*. Accessed: 2026-02-13. URL: <https://groq.com>.
- [7] IDC - International Data Corporation. *Growth Expected to Pause for AR/VR Headsets, according to IDC*. Rapp. tecn. Accessed: 2026-02-12. IDC, mar. 2025. URL: <https://www.idc.com/getdoc.jsp?containerId=prUS53278025>.
- [8] Meta AI. *Llama 3: The most capable openly available LLM to date*. Accessed: 2026-02-13. 2024. URL: https://www.llama.com/docs/model-cards-and-prompt-formats/llama3_3/.
- [9] Meta Platforms, Inc. *Compatibility Mode - Meta Horizon OS Documentation*. Accessed: 2026-02-12. Meta Platforms, Inc. 2024. URL: <https://developers.meta.com/horizon/documentation/native/android/os-compatibility-mode/>.
- [10] Meta Platforms, Inc. *Meta Horizon OS Discovery: Building Blocks for Unity*. Accessed: 2026-02-13. 2024. URL: <https://developers.meta.com/horizon/documentation/unity/bb-overview/>.
- [11] Meta Platforms, Inc. *Meta Voice SDK - Immersive Voice Commands*. <https://assetstore.unity.com/packages/tools/integration/meta-voice-sdk-immersive-voice-commands-264555>. Accessed: 2026-02-13. 2024.
- [12] Meta Platforms, Inc. *Meta XR Interaction SDK*. <https://assetstore.unity.com/packages/tools/integration/meta-xr-interaction-sdk-265014>. Accessed: 2026-02-12. 2024.

- [13] Ross E. Mitchell e Travas A. Young. «How Many People Use Sign Language? A National Health Survey-Based Estimate». In: *Journal of Deaf Studies and Deaf Education* 28.1 (2023). Accessed: 2026-02-11. URL: <https://academic.oup.com/jdsde/article/28/1/1/6845390>.
- [14] Oline Ranum et al. *3D-LEX v1.0: 3D Lexicons for American Sign Language and Sign Language of the Netherlands*. Accessed: 2026-02-13. 2024. URL: <https://arxiv.org/abs/2409.01901>.
- [15] Katherine D. Rogers et al. «Health outcomes in Deaf signing populations: A systematic review». In: *PLOS ONE* (2024). Accessed: 2026-02-11. URL: <https://pmc.ncbi.nlm.nih.gov/articles/PMC11020444/>.
- [16] Katherine D. Rogers et al. «Hearing loss in the working-age population: a systematic analysis from the Global Burden of Disease study 2021». In: *BMJ Open* (2025). Accessed: 2026-02-11. URL: <https://bmjopen.bmj.com/content/bmjopen/15/11/e103248.full.pdf>.
- [17] Serif Europe Ltd. *Affinity Creative Software Suite*. Accessed: 2026-02-13. 2024. URL: <https://www.affinity.studio/>.
- [18] Studio Galt. *Sign Language Mocap Archive: High-resolution 3D Motion Capture Data for ASL and SLN*. <https://github.com/StudioGalt/Sign-Language-Mocap-Archive>. Accessed: 2026-02-13. 2024.
- [19] Unity Technologies. *Unity User Manual 6.2*. Accessed: 2026-02-12. Unity Technologies. 2026. URL: <https://docs.unity3d.com/Manual/index.html>.
- [20] Unity Technologies. *XR, VR, and AR Game Development Software: Solutions and Impact*. Accessed: 2026-02-12. Unity Technologies. 2026. URL: <https://unity.com/solutions/xr/games>.
- [21] World Health Organization. *Deafness and hearing loss*. Accessed: 2026-02-11. 2024. URL: <https://www.who.int/news-room/fact-sheets/detail/deafness-and-hearing-loss>.
- [22] Zhi-qiang Zhang et al. «Global burden of hearing loss in people aged 60 years and older, 1990–2021: results from the global burden of disease study». In: *Frontiers in Public Health* (2024). Accessed: 2026-02-11. URL: <https://pmc.ncbi.nlm.nih.gov/articles/PMC12689307/>.